Solution: Design Add and Search Words Data Structure

Problem Statement

Design a data structure that supports the addition of new words and the ability to check if a string matches any previously added word.

Implement the `Solution` class:

- `Solution()` Initializes the object.
- `void addWord(word)` Inserts `word` into the data structure, making it available for future searches.
- `bool search(word)` Checks if there is any word in the data structure that matches `word`. The method returns `true` if such a match exists, otherwise returns `false`.

Note: In the search query `word`, the character `'.'` can represent any single letter, effectively serving as a wildcard character.

Examples

Example 1:

- **Input**:

```
["Solution", "addWord", "addWord", "search", "search"]
[[], ["apple"], ["banana"], ["apple"], ["....."]]
```

- **Expected Output**:

```
[-1, -1, -1, 1, 1]
```

- Justification: After adding the words "apple" and "banana", searching for "apple" will return `true` since "apple" is in the data structure. Searching for "....." will also return `true` as both "apple" and "banana" match the pattern.

Example 2:

- **Input**:

```
["Solution", "addWord", "addWord", "search", "search"]
[[], ["cat"], ["dog"], ["c.t"], ["d..g"]]
```

- **Expected Output**:

```
[-1, -1, -1, 1, 0]
```

- **Justification**: "c.t" matches "cat" and "d..g" doesn't matches "dog".

Example 3:

- **Input**:

```
["Solution", "addWord", "search", "search"]
[[], ["hello"], ["h.llo"], ["h...o"]]
```

- **Expected Output**:

```
[-1, -1, 1, 0]
```

- **Justification**: "h.llo" and "h...o" both match "hello".

Solution

The crux of the problem lies in efficiently inserting words and then searching for them, even if the query includes wildcards. To solve this, we utilize the trie (prefix tree) data structure. A trie is a tree-like structure that's useful for storing a dynamic set of strings, especially when the dataset involves large numbers of queries on prefixes of strings. Each node of the trie can represent a character of a word, and the path from the root node to any node represents the word stored up to that point. The key operation for the wildcard is a recursive search, which allows us to explore multiple paths in the trie when we encounter the wildcard character.

1. Trie Data Structure: Every node of the trie contains multiple child nodes (one for each character of the alphabet). We start with a root node that represents an empty string. Each level of the trie represents the next character of a word.

2. Adding a Word: To insert a word into our trie, we begin at the root and traverse down the trie based on the characters in the word. If a particular character doesn't have a corresponding child node in the current node, we create a new child node for that character. Once we've processed every character of the word, we mark the final node as the end of a valid word.

3. Searching: Searching for a word is similar to inserting, but with an additional consideration for the wildcard character ('.'). If we encounter a '.', we must consider all child nodes of the current node and recursively continue our search from each of them. If any of the paths result in a match, we return `true`. If we reach the end of a word without encountering any mismatches or premature ends, we've found a valid word in our trie.

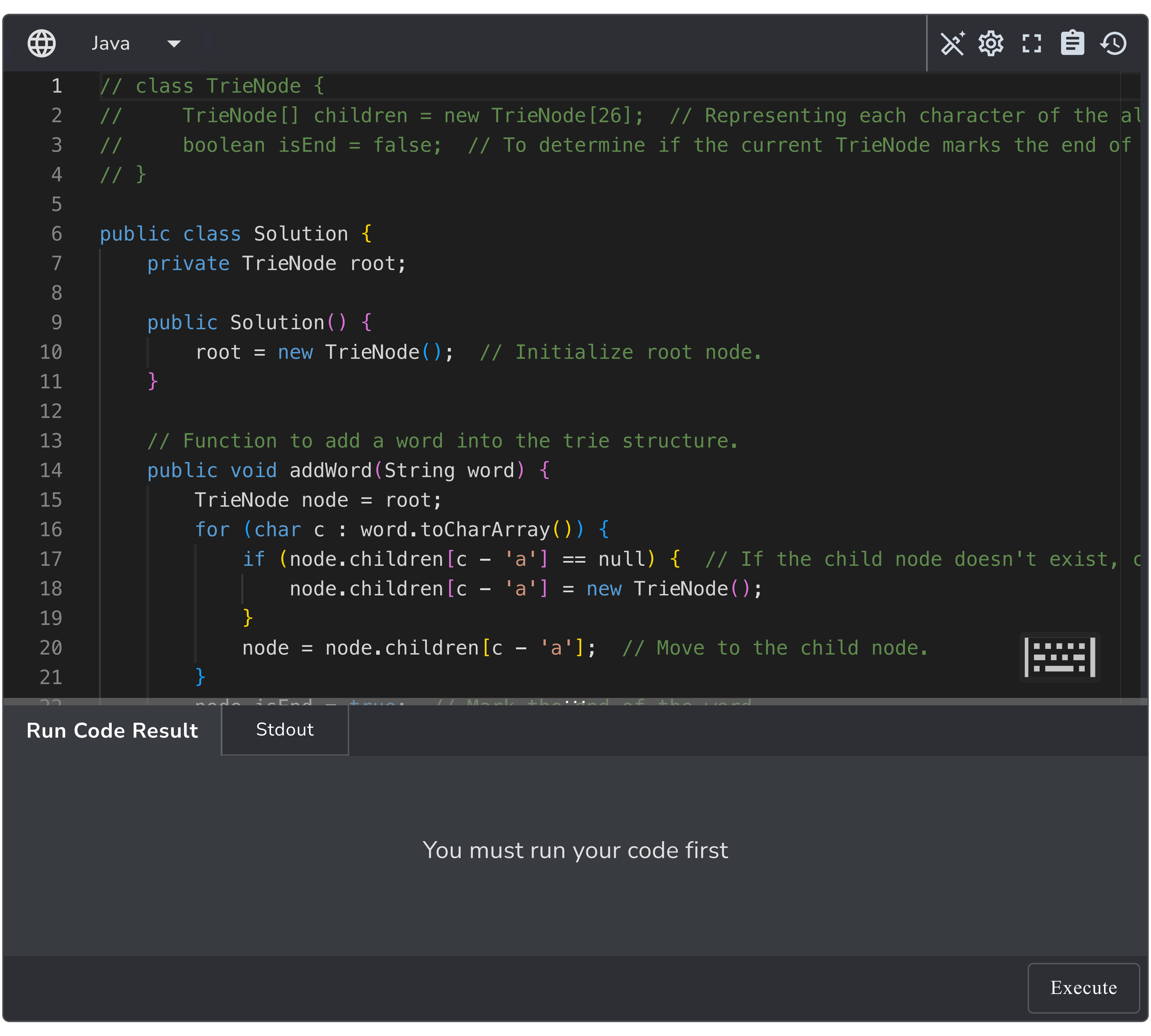
This trie-based approach ensures efficient operations for both inserting and searching for words. In cases without wildcards, the search operation can be performed in linear time relative to the word's length. However, with wildcards, the time complexity might increase, but the trie structure still ensures that we do this efficiently.

Algorithm Walkthrough

Given the word "apple" to insert and then search for ".....":

1. Start at the root node.
2. For inserting "apple":
   - At 'a', move down or create a node if it doesn't exist.
   - At 'p', move down or create.
   - Do the same for the next 'p', 'l', and 'e'.
   - Mark the last node (for 'e') as the end of a word.
3. For searching ".....":
   - At the first '.', check all child nodes and continue.
   - Repeat for each '.'.
   - If any path leads to a node that represents the end of a word, return `true`.

Code



```java
// class TrieNode {
//     TrieNode[] children = new TrieNode[26];  // Representing each character of the al
//     boolean isEnd = false;  // To determine if the current TrieNode marks the end of
// }

public class Solution {
    private TrieNode root;

    public Solution() {
        root = new TrieNode();  // Initialize root node.
    }

    // Function to add a word into the trie structure.
    public void addWord(String word) {
        TrieNode node = root;
        for (char c : word.toCharArray()) {
            if (node.children[c - 'a'] == null) {  // If the child node doesn't exist, c
                node.children[c - 'a'] = new TrieNode();
            }
            node = node.children[c - 'a'];  // Move to the child node.
        }
```

You must run your code first

Complexity Analysis

- **Time Complexity**:
  - **Insertion (addWord)**: O(n), where n is the length of the word. This is because each insertion operation runs in linear time with respect to the length of the word.
  - **Search:** O(n * m) in the worst case, where n is the length of the word and m is the total number of nodes in the Trie. This happens when the search word contains dots ('.'). However, for words without dots, the search is O(n).
- **Space Complexity**: O(m * n), where m is the total number of Trie nodes and n is the average number of characters in the words. Each Trie node has up to 26 children (for each letter of the alphabet). In the worst case, when no nodes are shared, the space complexity is O(m * n).

← Previous
Design Add And Search Words Data Structure (Medium)

Next →
Introduction To Greedy Algorithm

☐ Mark as Completed